BECE102L-DIGITAL SYSTEMS DESIGN

Module-4

Design of data path circuits

by

Dr. Rajkishor Kumar (Ph.D. | IIT, Dhanbad)
Assistant Professor Senior Grade, School of Electronics Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email Id/ Institute ID: rajkishorkumar88@gmail.com/ rajkishor.kumar@vit.ac.in
Personal Website: https://sites.google.com/site/rajkishormicrowave

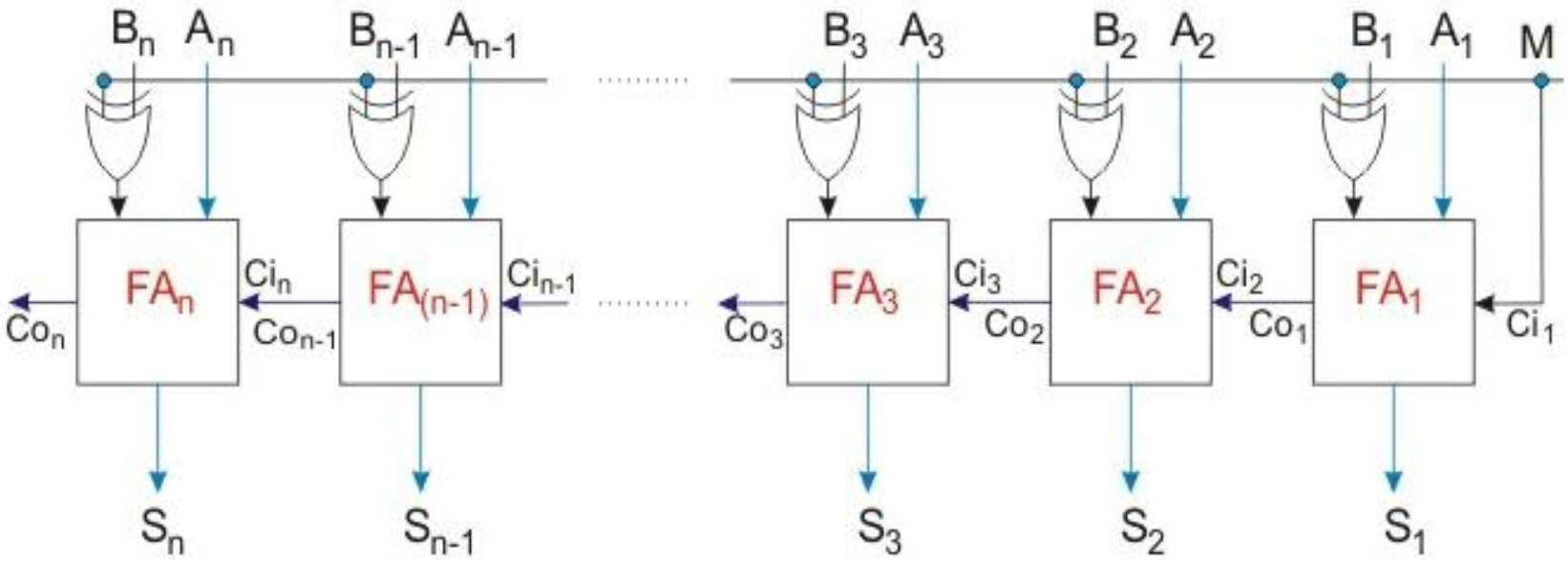
.....

Module: 4 Design of data path circuits

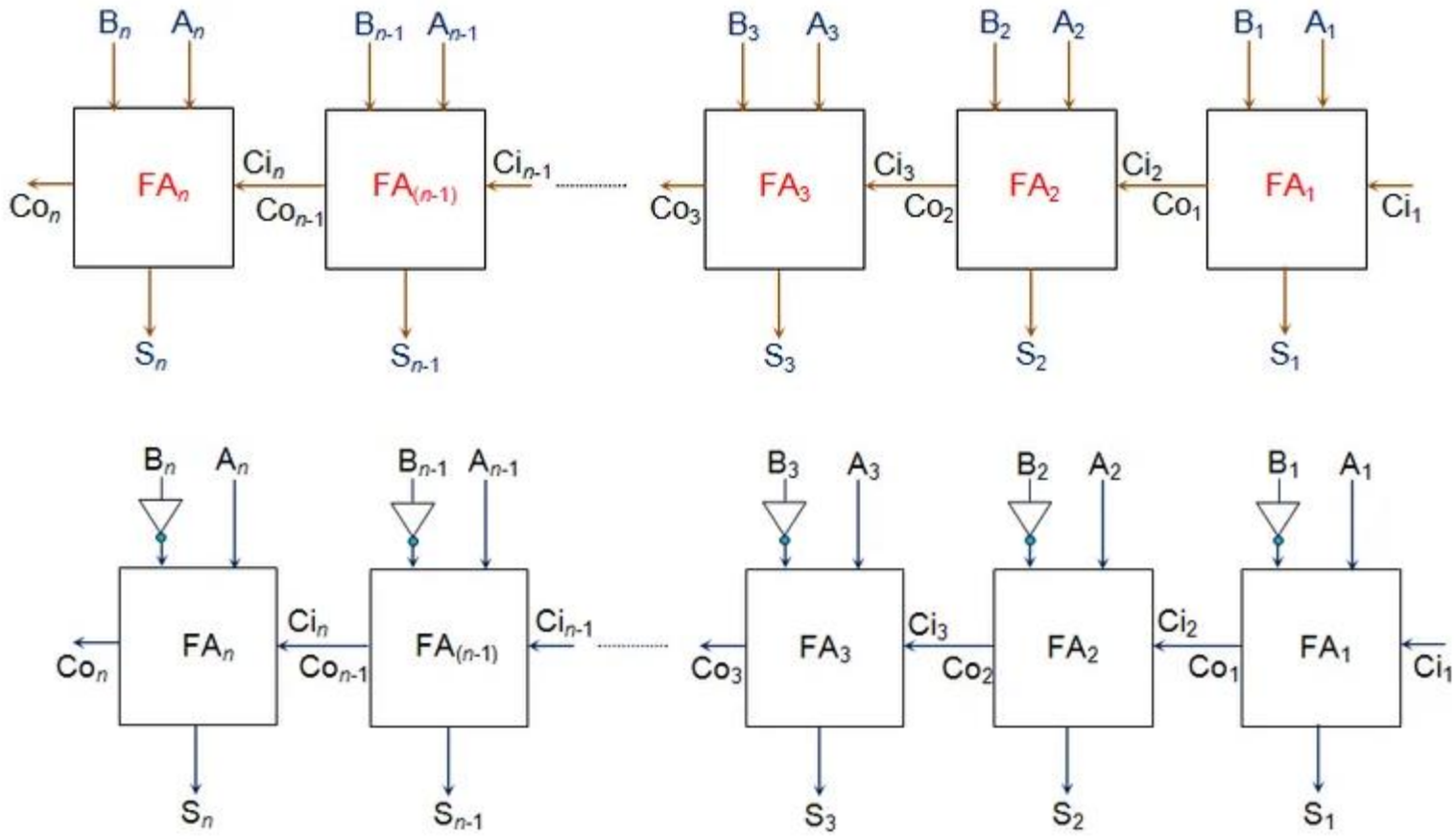
Module:4 Design of data path circuits 6 hours

N-bit Parallel Adder/Subtractor, Carry Look Ahead Adder, Unsigned Array Multiplier, Booth Multiplier, 4-Bit Magnitude comparator. Modeling of data path circuits using Verilog HDL

N-bit Parallel Adder/Subtractor



N Bit Adder and Subtractor



***n*-bit Parallel Subtractor Circuit Realized Using *n* full Adders**

N bit Adder code

```
module N_bit_adder(input1,input2,answer);
    parameter N=32;
              input [N-1:0] input1,input2;
              output [N-1:0] answer;
              wire  carry_out;
              wire [N-1:0] carry;
    genvar i;
      generate
                for(i=0;i<N;i=i+1)
                   begin: generate_N_bit_Adder
                        if(i==0)
                    half_adder f(input1[0],input2[0],answer[0],carry[0]);
                else
            full_adder f(input1[i],input2[i],carry[i-1],answer[i],carry[i]);
                end
                    assign carry_out = carry[N-1];
       endgenerate
endmodule
```

```
module half_adder(x,y,s,c);
   input x,y;
   output s,c;
   assign s=x^y;
   assign c=x&y;
endmodule // half adder


module full_adder(x,y,c_in,s,c_out);
   input x,y,c_in;
   output s,c_out;
 assign s = (x^y) ^ c_in;
 assign c_out = (y&c_in)| (x&y) | (x&c_in);
endmodule // full_adder
```

Sample testbench

```
module tb_N_bit_adder;
 // Inputs
 reg [31:0] input1;
 reg [31:0] input2;
 // Outputs
 wire [31:0] answer;

 // Instantiate the Unit Under Test (UUT)
 N_bit_adder uut (
  .input1(input1),
  .input2(input2),
  .answer(answer)
 );

 initial begin
  // Initialize Inputs
  input1 = 1209;
  input2 = 4565;
  #100;
  // Add stimulus here
 end

endmodule
```

Verilog for full adder

```
module fulladd(a,b,carryin,sum,carryout);
   input a, b, carryin; /* add these bits*/
   output sum, carryout; /* results */

   assign {carryout, sum} = a + b + carryin;
                  /*  compute the sum and carry */
   endmodule
```

Verilog for ripple-carry adder

```
module nbitfulladd(a,b,carryin,sum,carryout)
    input [7:0] a, b; /* add these bits */
    input carryin; /* carry in*/
    output [7:0] sum; /* result */
    output carryout;
    wire [7:1] carry; /* transfers the carry between bits */

    fulladd a0(a[0],b[0],carryin,sum[0],carry[1]);
    fulladd a1(a[1],b[1],carry[1],sum[1],carry[2]);
…
    fulladd a7(a[7],b[7],carry[7],sum[7],carryout]);
endmodule
```

Ref the code :

Ripple carry adder code reference from textbook

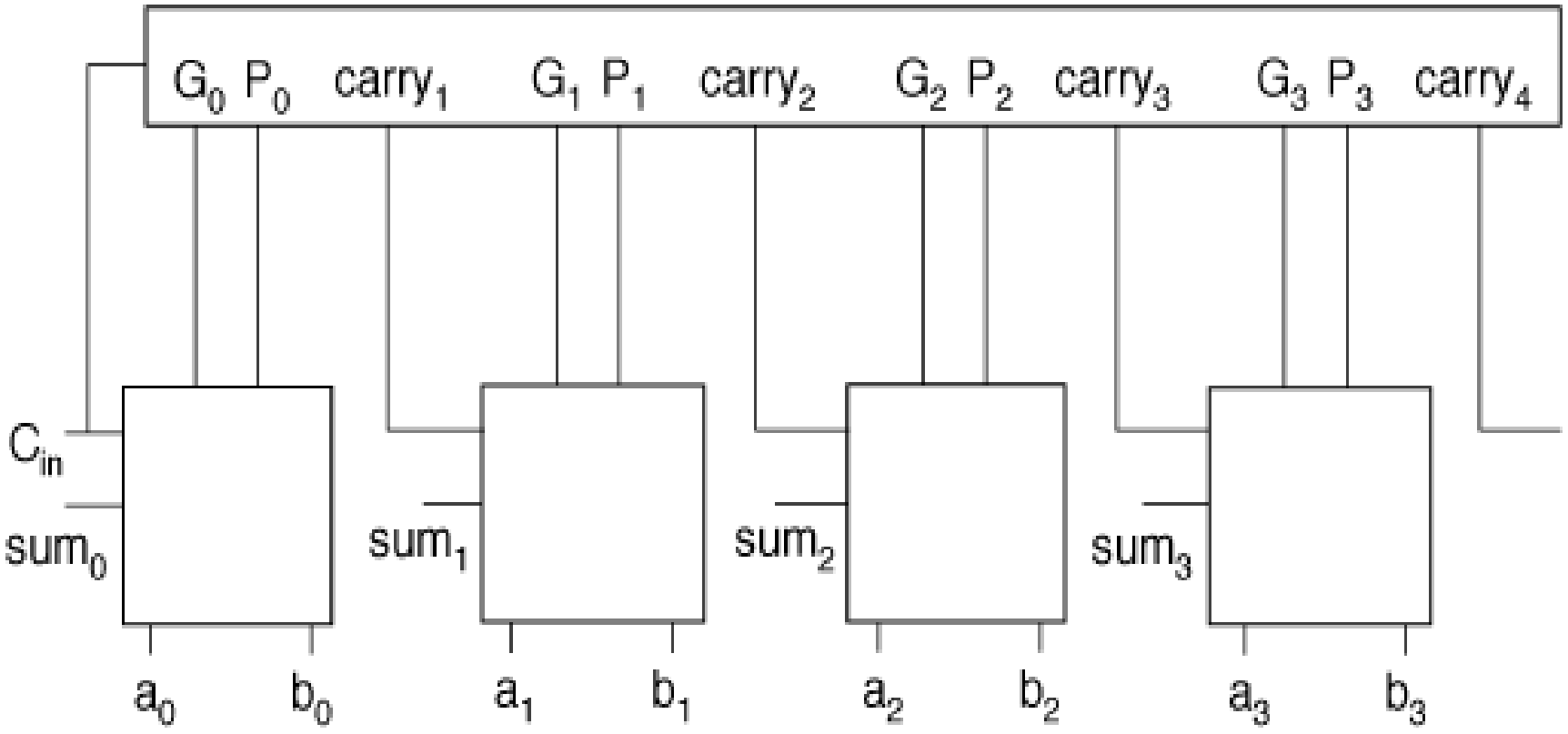
Carry-lookahead adder

- First compute carry propagate, generate:
  - $P_i = a_i + b_i$
  - $G_i = a_i b_i$
- Compute sum and carry from P and G:
  - $s_i = c_i$ XOR P_i XOR G_i
  - $c_{i+1} = G_i + P_i c_i$

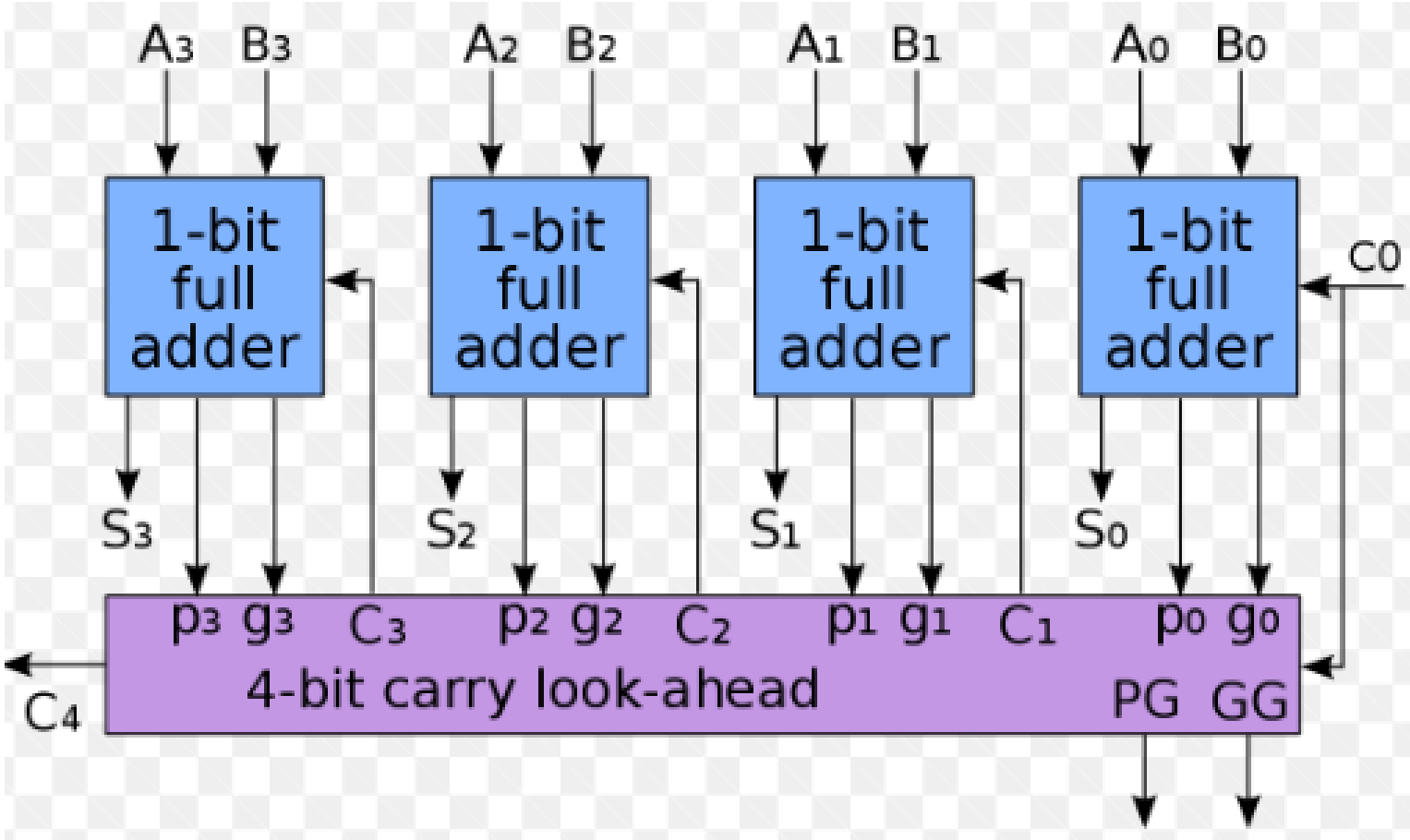
Carry-lookahead expansion

- Can recursively expand carry formula:
  - $c_{i+1} = G_i + P_i(G_{i-1} + P_{i-1}c_{i-1})$
  - $c_{i+1} = G_i + P_iG_{i-1} + P_iP_{i-1}(G_{i-2} + P_{i-1}c_{i-2})$
- Expanded formula does not depend on intermerdiate carries.
- Allows carry for each bit to be computed independently.

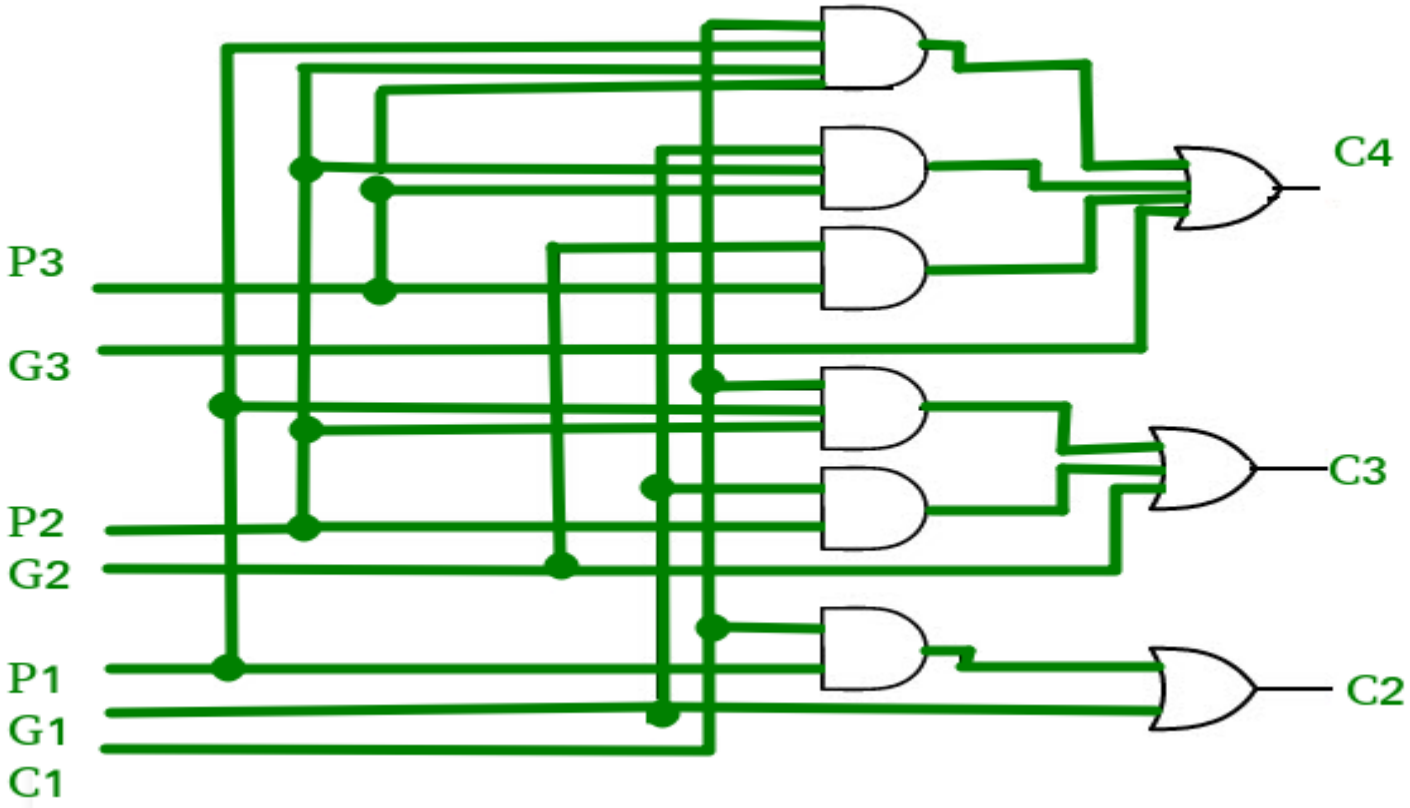
Depth-4 carry-lookahead



Carry Look Ahead Adder



CLA



Generate and Propagate

$$G[i] = A[i].B[i]$$
$$P[i] = A[i] \oplus B[i]$$
$$C[i] = G[i] + P[i].C[i-1]$$
$$S[i] = P[i] \oplus C[i-1]$$

$$G[i] = A[i].B[i]$$
$$P[i] = A[i] + B[i]$$
$$C[i] = G[i] + P[i].C[i-1]$$
$$S[i] = A[i] \oplus B[i] \oplus C[i-1]$$

Two methods to develop C[i] and S[i].

Verilog for carry-lookahead carry block

```
module carry_block(a,b,carryin,carry);
    input [3:0] a, b; /* add these bits*/
    input carryin; /* carry into the block */
    output [3:0] carry; /* carries for each bit in the block */
    wire [3:0] g, p; /* generate and propagate */

    assign g[0] = a[0] & b[0]; /* generate 0 */
    assign p[0] = a[0] ^ b[0]; /* propagate 0 */
    assign g[1] = a[1] & b[1]; /* generate 1 */
    assign p[1] = a[1] ^ b[1]; /* propagate 1 */
...
    assign carry[0] = g[0] | (p[0] & carryin);
    assign carry[1] = g[1] | p[1] & (g[0] | (p[0] & carryin));
    assign carry[2] = g[2] | p[2] & (g[1] | p[1] & (g[0] | (p[0] & carryin)));
    assign carry[3] = g[3] | p[3] & (g[2] | p[2] & (g[1] | p[1] & (g[0] | (p[0] & carryin))));
```

$$c_{i+1} = G_i + P_i(G_{i-1} + P_{i-1}C_{i-1})$$

- endmodule

Verilog for carry-lookahead sum unit

```
module sum(a,b,carryin,result);
  input a, b, carryin;   /* add these bits*/
  output result;         /* sum */

  assign result = a ^ b ^ carryin;
                /*  compute the sum  */
endmodule
```

Verilog for carry-lookahead adder

- module carry_lookahead_adder(a,b,carryin,sum,carryout);
  input [15:0] a, b; /* add these together */
  input carryin;
  output [15:0] sum; /* result */
  output carryout;
  wire [16:1] carry; /* intermediate carries */

  assign carryout = carry[16]; /* for simplicity */
  /* build the carry-lookahead units */
  carry_block b0(a[3:0],b[3:0],carryin,carry[4:1]);
  carry_block b1(a[7:4],b[7:4],carry[4],carry[8:5]);
  carry_block b2(a[11:8],b[11:8],carry[8],carry[12:9]);
  carry_block b3(a[15:12],b[15:12],carry[12],carry[16:13]);
  /* build the sum */
  sum a0(a[0],b[0],carryin,sum[0]);
  sum a1(a[1],b[1],carry[1],sum[1]);

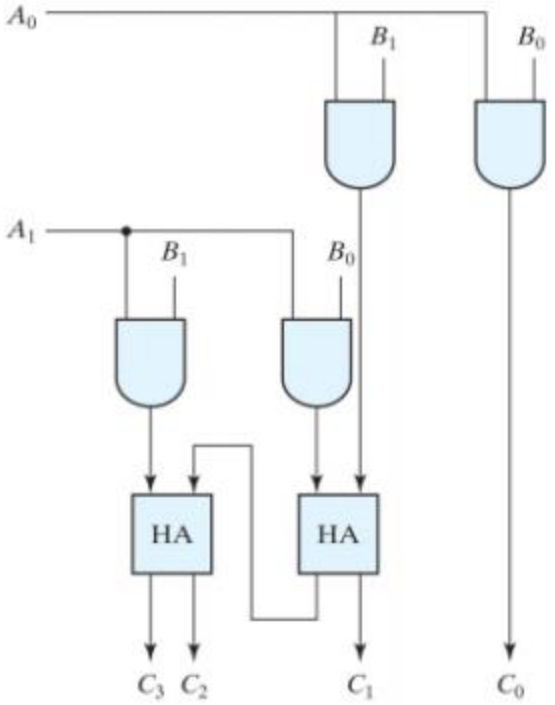
…

  sum a15(a[15],b[15],carry[15],sum[15]);

endmodule

Unsigned 2x2 Array Multiplier

		B_1	B_0
		A_1	A_0
		A_0B_1	A_0B_0
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0



A1 A0

×

B1 B0

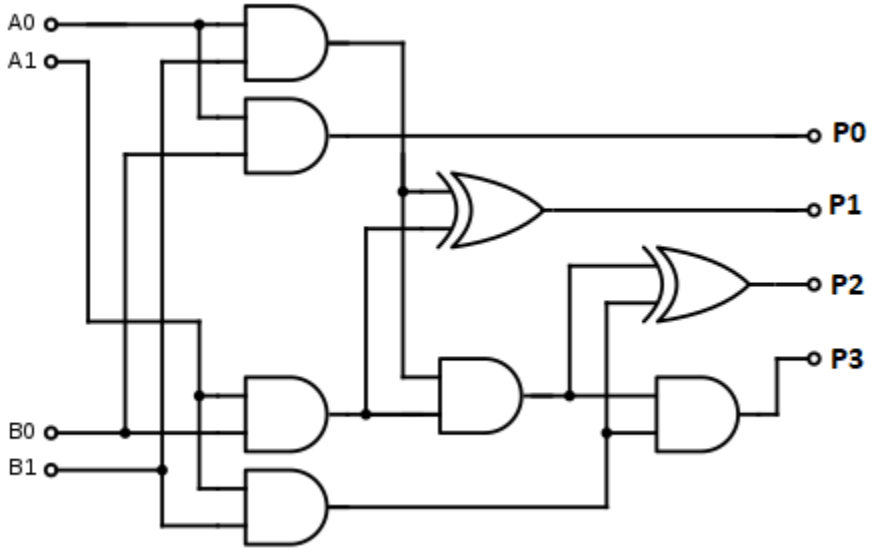
B0 A1	A0 B0	← Partial products
B1A1	B1 A0	
P3	P2	P1 P0

P0 = A0 B0

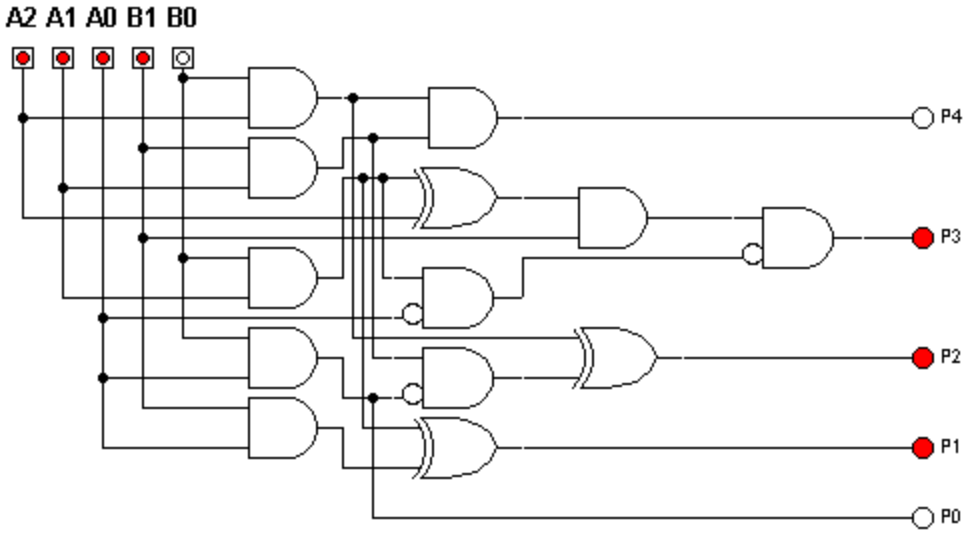
P1 = B0 A1 + B1 A0

P2 = B1A1 + Carryout of P1

P3 = Carryout of P2



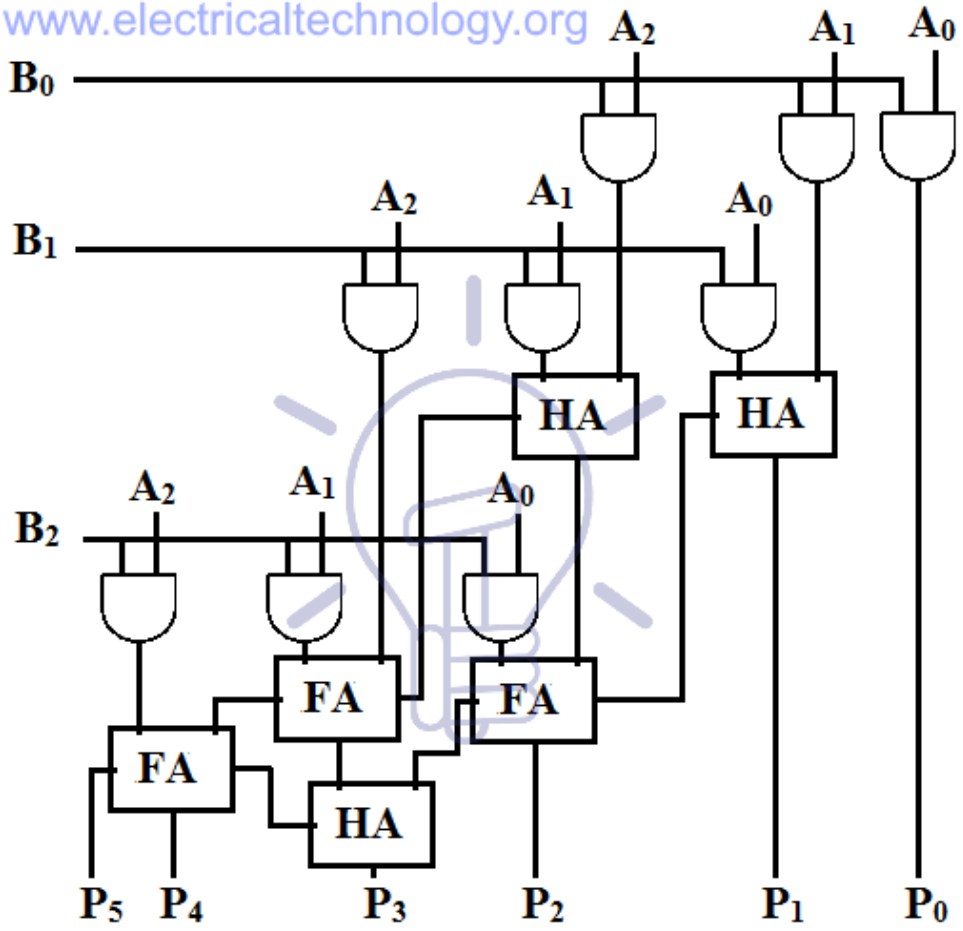
3 x 2 / 2 x 3 Multiplier circuit



	A2	A1	A0
		B1	B0
	A2B0	A1B0	A0B0
A2B1	A1B1	A0B1	X

A2B1 +C · A1B1+C+ ·A2B0 A0B1+A1B0 A0B0

3 X 3 Multiplier

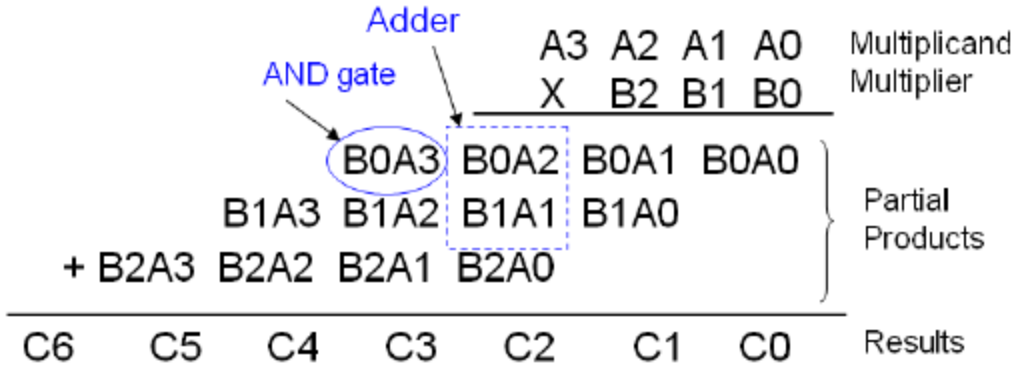


3-bit by 3-bit

		a_2	a_1	a_0
x		b_2	b_1	b_0
		a_2b_0	a_1b_0	a_0b_0
	a_2b_1	a_1b_1	a_0b_1	
a_2b_2	a_1b_2	a_0b_2		

		A2 B2	A1 B1	A0 B0
		A2B0	A1B0	A0B0
	A2B1	A1B1	A0B1	X
A2B2	A1B2	A0B2	X	X
A2B2+C+C	A2B1+A1B2 +A2B2+C+C	A1B1+C+ A0B2+A2B0	A0B1+A1B0	A0B0

4 x 3 Multiplier



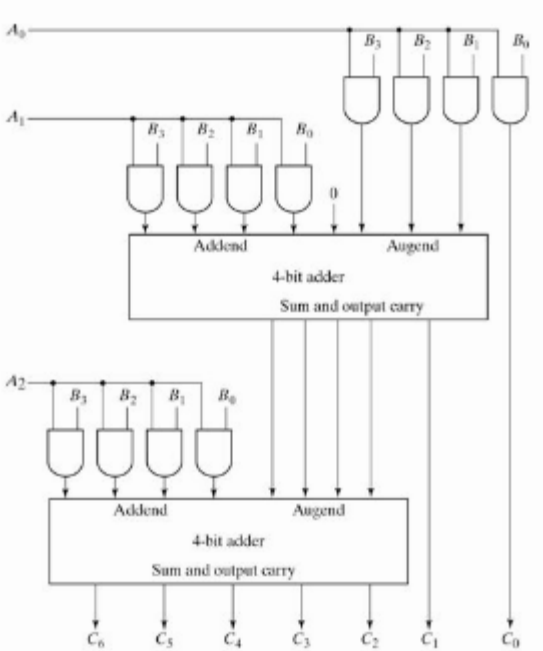
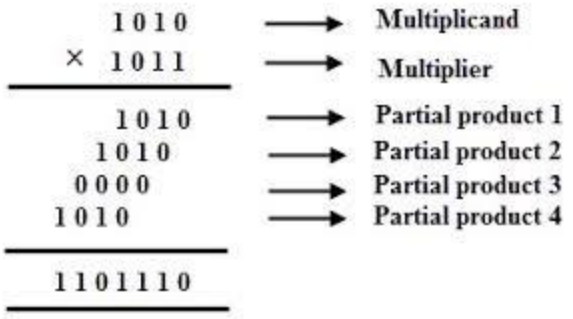


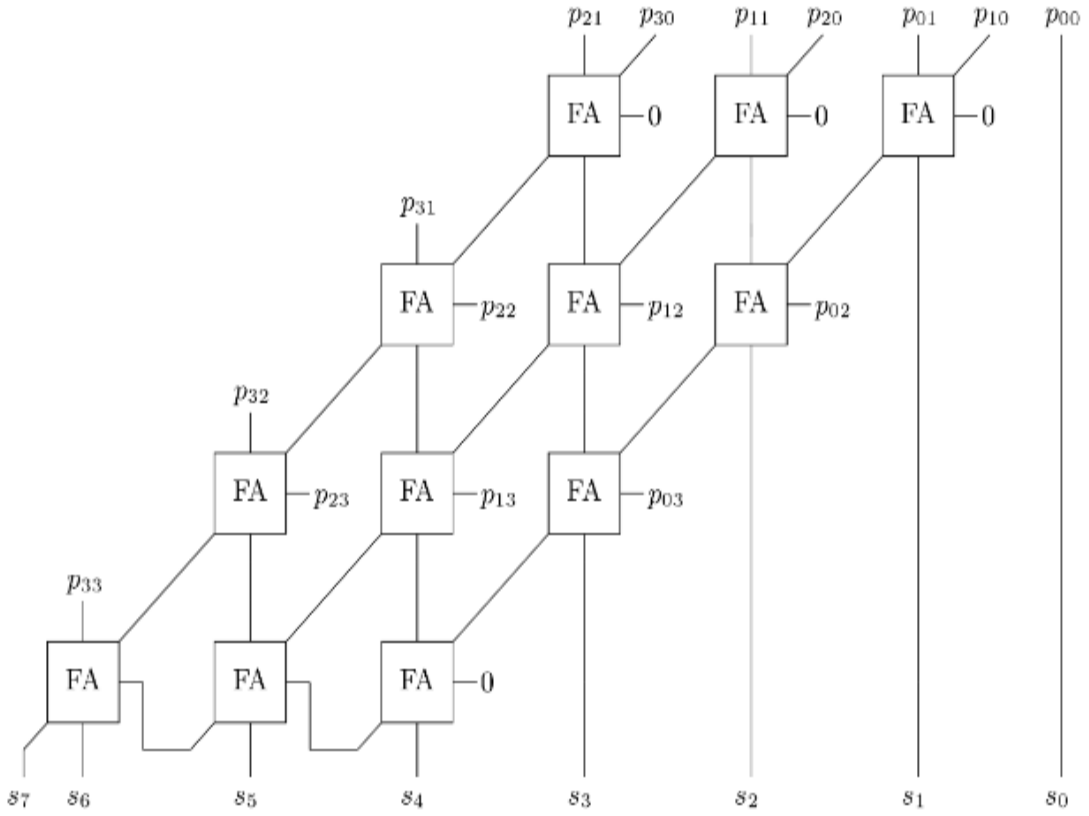
Fig. 4-16 4-Bit by 3-Bit Binary Multiplier

Unsigned 4x4 Array Multiplier

$$
\begin{array}{cccccccc}
 & & & & a_3 & a_2 & a_1 & a_0 \\
 & & & \times & b_3 & b_2 & b_1 & b_0 \\
\hline
 & & & & p_{30} & p_{20} & p_{10} & p_{00} \\
 & & & p_{31} & p_{21} & p_{11} & p_{01} & \times \\
 & & p_{32} & p_{22} & p_{12} & p_{02} & \times & \times \\
 & p_{33} & p_{23} & p_{13} & p_{03} & \times & \times & \times \\
\hline
s_7 & s_6 & s_5 & s_4 & s_3 & s_2 & s_1 & s_0
\end{array}
$$

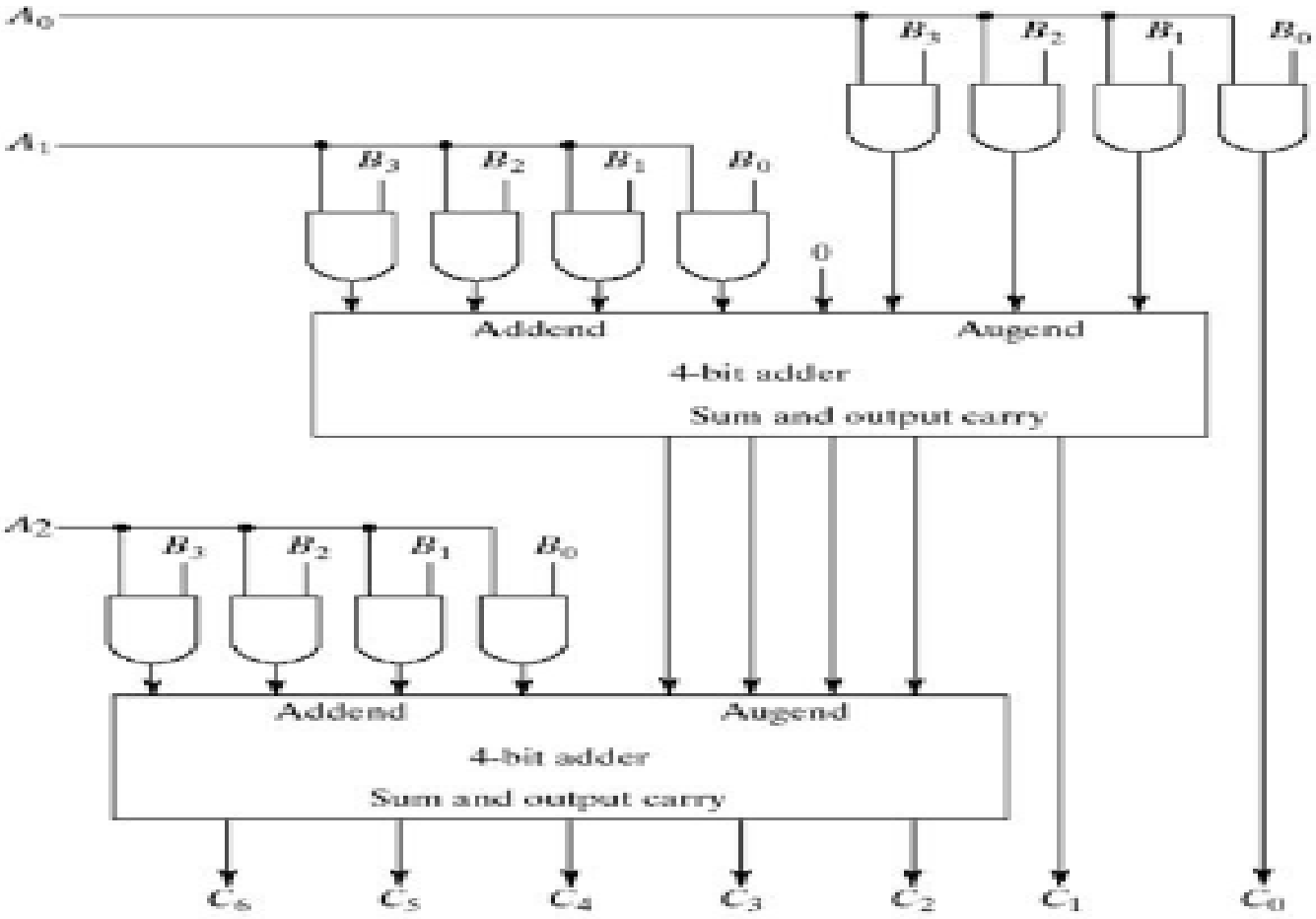
Multiplication of two 4-bit unsigned numbers



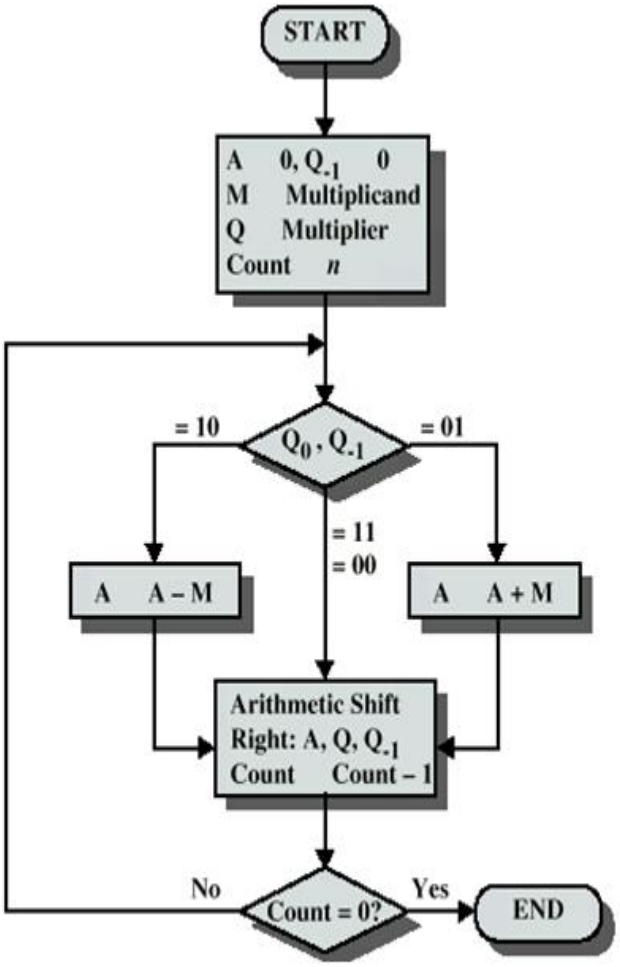


Unsigned Array Multiplier

4 x4 Multiplier

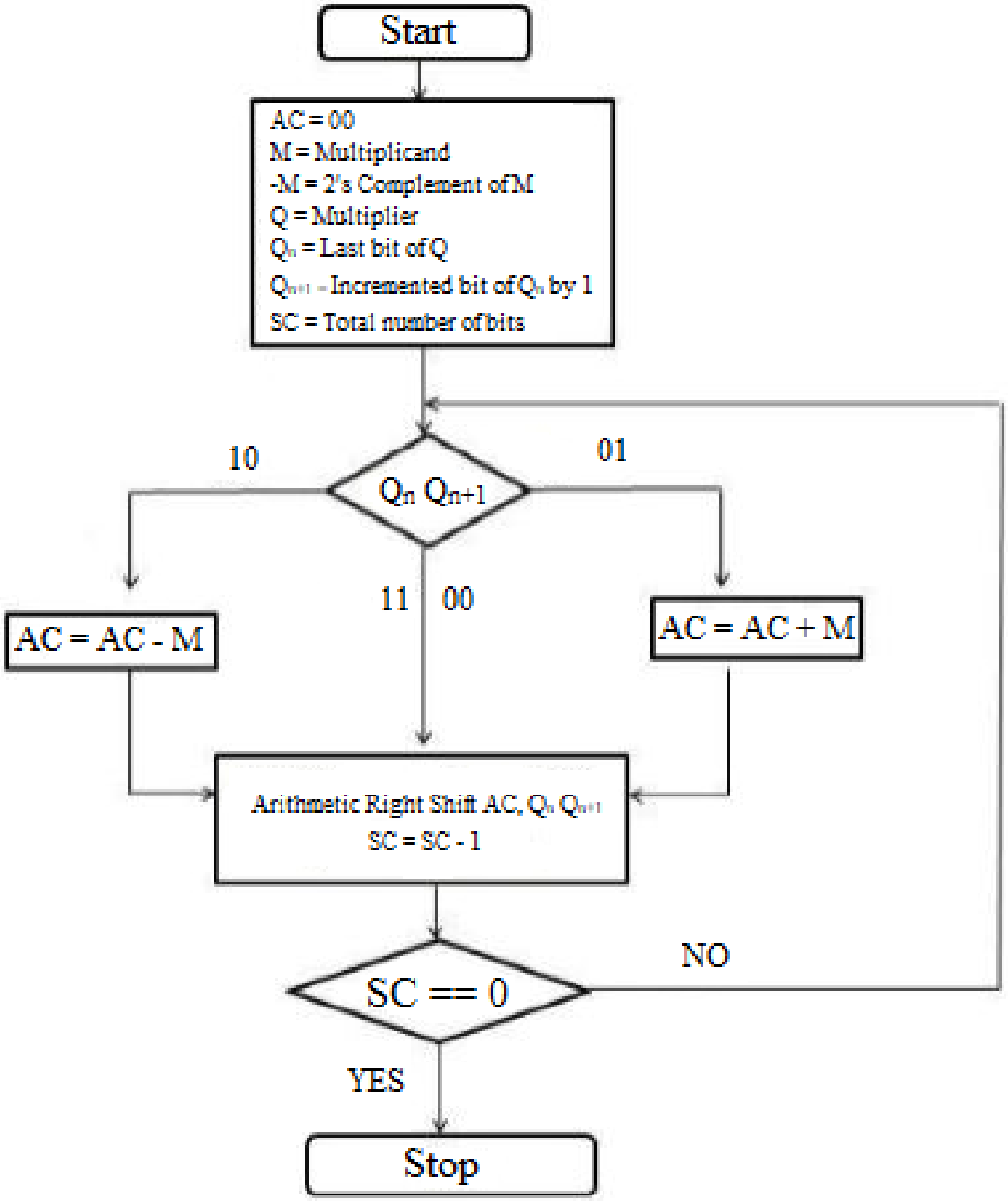


Booth Multiplier



Flow Chart Of Booth Multiplier Algorithm

1. Set the Multiplicand and Multiplier binary bits as M and Q, respectively.
2. Initially, we set the AC and Q_{n+1} registers value to 0.
3. SC represents the number of Multiplier bits (Q), and it is a sequence counter that is continuously decremented till equal to the number of bits (n) or reached to 0.
4. A Qn represents the last bit of the Q, and the Q_{n+1} shows the incremented bit of Qn by 1.
5. On each cycle of the booth algorithm, Q_n and Q_{n+1} bits will be checked on the following parameters as follows:
   i. When two bits Q_n and Q_{n+1} are 00 or 11, we simply perform the arithmetic shift right operation (ashr) to the partial product AC. And the bits of Qn and Q_{n+1} is incremented by 1 bit.
   ii. If the bits of Q_n and Q_{n+1} is shows to 01, the multiplicand bits (M) will be added to the AC (Accumulator register). After that, we perform the right shift operation to the AC and QR bits by 1.
   iii. If the bits of Q_n and Q_{n+1} is shows to 10, the multiplicand bits (M) will be subtracted from the AC (Accumulator register). After that, we perform the right shift operation to the AC and QR bits by 1.
6. The operation continuously works till we reached n - 1 bit in the booth algorithm.
7. Results of the Multiplication binary bits will be stored in the AC and QR registers.


Start
AC = 00
M = Multiplicand
-M = 2's Complement of M
Q = Multiplier
Qn = Last bit of Q
Qn+1 = Incremented bit of Qn by 1
SC = Total number of bits
10
01
Qn Qn+1
11
00
AC = AC - M
AC = AC + M
Arithmetic Right Shift AC, Qn Qn+1
SC = SC - 1
NO
SC == 0
YES
Stop

1. Initialize the multiplicand (A), multiplier (B), accumulator (AC), and shift counter (SC).
2. Set the flag (Qn) to 0.
3. Right-shift the multiplier and Qn by one bit.
4. Examine the two least significant bits of the multiplier and Qn (bits 1 and 0). Depending on their values, perform one of three actions:
   a. If the bits are 01, subtract the multiplicand from the accumulator.
   b. If the bits are 10, add the multiplicand to the accumulator.
   c. If the bits are 00 or 11, take no action.
5. Right-shift the accumulator by one bit.
6. Decrease the shift counter.
7. Repeat steps 3-6 until the shift counter reaches zero.
8. The final product is obtained by concatenating the accumulator and the shifted multiplier ([AC, B]).

Multiplicand = 7 (M) = 0111

Multiplier = 3 (Q) = 0011

Accumulator = 0(A) = 0000

Accumulator (A)	Multiplier(Q)	Test Bit(T)	Operation	Operation Number
0000	0011	0	Subtraction (A <= A+M'+1)	I
1001	0011	0	Shift Right	
1100	1001	1	Shift Right	II
1110	0100	1	Addition (A <= A+M)	III
0101	0100	1	Shift Right	
0010	1010	0	Shift Right	IV
0001	0101	0	Finish	

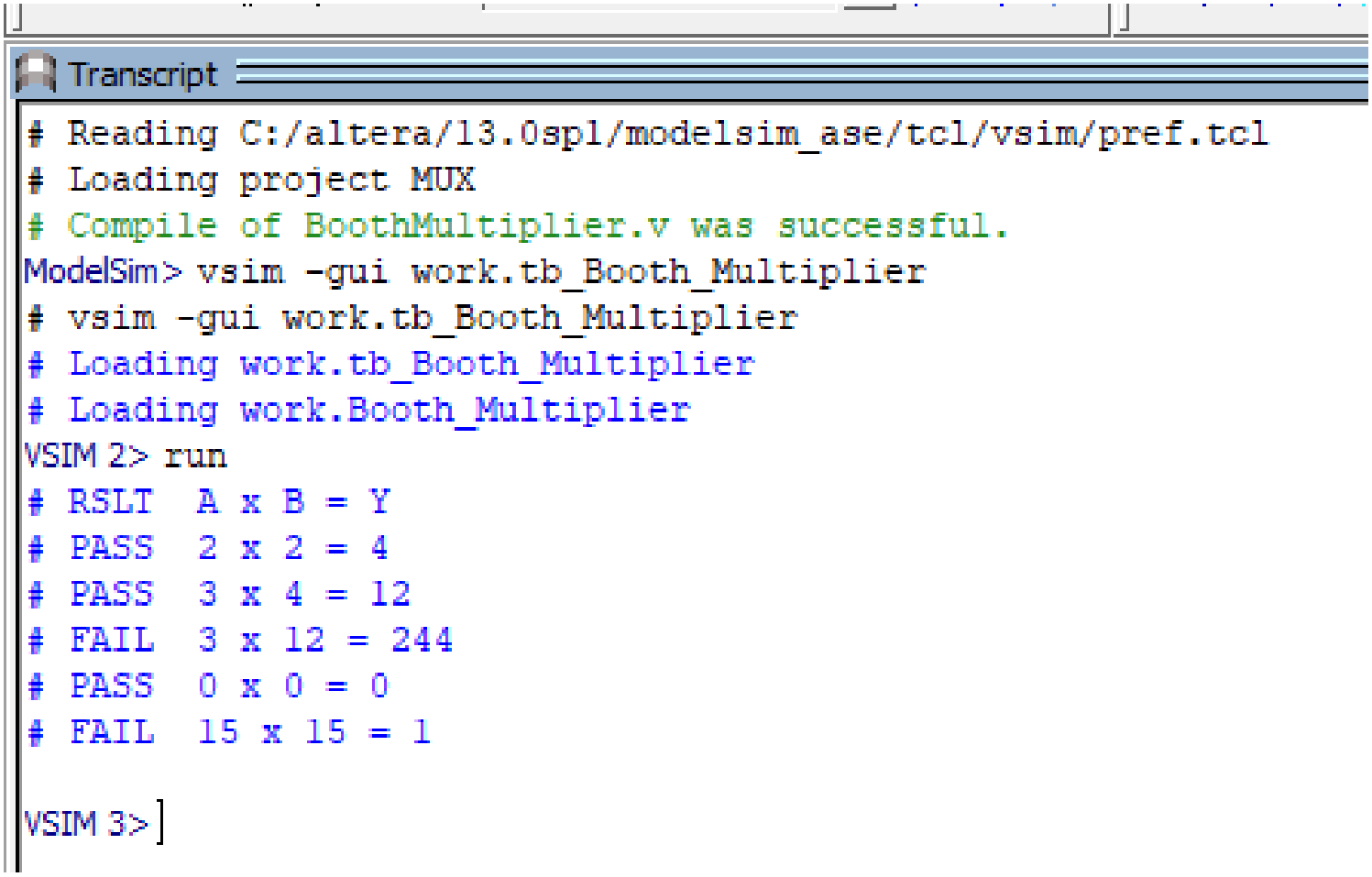
```
1  module Booth_Multiplier(PRODUCT, A, B);
2    output reg signed [7:0] PRODUCT;
3    input signed [3:0] A, B;
4
5    reg [1:0] temp;
6    integer i;
7    reg e;
8    reg [3:0] B1;
9
10   always @(A,B)
11   begin
12     PRODUCT = 8'd0;
13     e = 1'b0;
14     B1 = -B;
15
16     for (i=0; i<4; i=i+1)
17     begin
18       temp = { A[i], e };
19       case(temp)
20         2'd2 : PRODUCT[7:4] = PRODUCT[7:4] + B1;
21         2'd1 : PRODUCT[7:4] = PRODUCT[7:4] + B;
22       endcase
23       PRODUCT = PRODUCT >> 1;
24       PRODUCT[7] = PRODUCT[6];
25       e=A[i];
```

```
26
27      end
28    end
29
30  endmodule
31
```

Testbench for Booth Multiplier

```
34 module tb_Booth_Multiplier;
35   wire [7:0] Y;
36   reg [3:0] A, B;
37
38   Booth_Multiplier dut(Y, A, B);
39
40   initial
41   begin
42
43     $display("RSLT\tA x B = Y");          //Results comes in signed fixed number
44     A = 2; B = 2; #10;                   //but if it comes in unsigned number so
45     if ( Y == 4 )                       //the answer is also correst but in that
46       $display("PASS\t%p x %p = %p",A,B,Y);  //case transcript shows fail.
47     else
48       $display("FAIL\t%p x %p = %p",A,B,Y);
49
50
51     A = 3; B = 4; #10;
52     if ( Y == 12 )
53       $display("PASS\t%p x %p = %p",A,B,Y);
54     else
55       $display("FAIL\t%p x %p = %p",A,B,Y);
56
```

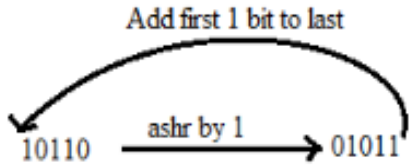
```
57      A = 3; B = -4; #10;
58      if ( Y == -12 )
59        $display("PASS\t%p x %p = %p",A,B,Y);
60      else
61        $display("FAIL\t%p x %p = %p",A,B,Y);
62
63
64      A = 0; B = 0; #10;
65      if ( Y == 0 )
66        $display("PASS\t%p x %p = %p",A,B,Y);
67      else
68        $display("FAIL\t%p x %p = %p",A,B,Y);
69
70
71      A = 15; B = 15; #10;
72      if ( Y == 225 )
73        $display("PASS\t%p x %p = %p",A,B,Y);
74      else
75        $display("FAIL\t%p x %p = %p",A,B,Y);
76
77    end
78
79  endmodule
```


Transcript
Reading C:/altera/13.0sp1/modelsim_ase/tcl/vsim/pref.tcl
Loading project MUX
Compile of BoothMultiplier.v was successful.
ModelSim> vsim -gui work.tb_Booth_Multiplier
vsim -gui work.tb_Booth_Multiplier
Loading work.tb_Booth_Multiplier
Loading work.Booth_Multiplier
VSIM 2> run
RSLT A x B = Y
PASS 2 x 2 = 4
PASS 3 x 4 = 12
FAIL 3 x 12 = 244
PASS 0 x 0 = 0
FAIL 15 x 15 = 1
VSIM 3>

Shift operation

1. RSC (Right Shift Circular)

It shifts the right-most bit of the binary number, and then it is added to the beginning of the binary bits.



2. RSA (Right Shift Arithmetic)

It adds the two binary bits and then shift the result to the right by 1-bit position.

Example: 0100 + 0110 => 1010, after adding the binary number shift each bit by 1 to the right and put the first bit of resultant to the beginning of the new bit.

Example: Multiply the two numbers 7 and 3 by using the Booth's multiplication algorithm.

Ans. Here we have two numbers, 7 and 3.

First of all, we need to convert 7 and 3 into binary numbers like 7 = (0111) and 3 = (0011).

Now set 7 (in binary 0111) as multiplicand (M) and 3 (in binary 0011) as a multiplier (Q). And SC (Sequence Count) represents the number of bits, and here we have 4 bits, so set the SC = 4.

Also, it shows the number of iteration cycles of the booth's algorithms and then cycles run SC = SC - 1 time.

Q_n	Q_{n+1}	M = (0111) M' + 1 = (1001) & Operation	AC	Q	Q_{n+1}	SC
1	0	Initial	0000	0011	0	4
		Subtract (M' + 1)	1001			
			1001			
		Perform Arithmetic Right Shift operations (ashr)	1100	1001	1	3
1	1	Perform Arithmetic Right Shift operations (ashr)	1110	0100	1	2
0	**1**	Addition (A + M)	0111			
			0101	0100		
		Perform Arithmetic right shift operation	0010	1010	0	1
0	0	Perform Arithmetic right shift operation	**0001**	**0101**	0	0

- The numerical example of the Booth's Multiplication Algorithm is 7 x 3 = 21 and the binary representation of 21 is 10101.
- Here, we get the resultant in binary 00010101. Now we convert it into decimal, as $(000010101)_{10}$ = 2*4 + 2*3 + 2*2 + 2*1 + 2*0 => 21

```
module boothmul(  input [7:0]a,b, output [15:0] c );
        wire  [7:0]Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7;   wire  [7:0] A1,A0,A3,A2;
        wire  [7:0] A4,A5,A6,A7;       wire [7:0] q0; wire qout;
        booth_substep step1(8'b00000000,a,1'b0,b,A1,Q1,q0[1]);
        booth_substep step2(A1,Q1,q0[1],b,A2,Q2,q0[2]);
        booth_substep step3(A2,Q2,q0[2],b,A3,Q3,q0[3]);
        booth_substep step4(A3,Q3,q0[3],b,A4,Q4,q0[4]);
        booth_substep step5(A4,Q4,q0[4],b,A5,Q5,q0[5]);
        booth_substep step6(A5,Q5,q0[5],b,A6,Q6,q0[6]);
        booth_substep step7(A6,Q6,q0[6],b,A7,Q7,q0[7]);
        booth_substep step8(A7,Q7,q0[7],b,c[15:8],c[7:0],qout);
endmodule
module booth_substep( input [7:0]A,Q, input  q0, input [7:0]M, output
[7:0]A_,Q_, output  Q_1 );
 wire [7:0] sum, difference;
 assign {A_,Q_,Q_1} = ((Q[0] < q0) ? {sum[7],sum,Q} : ((Q[0] > q0)  ?
                      {difference[7], difference, Q} : {A[7],A,Q}) )  ;
 ALU sub( A,~M,1'b1,difference);  ALU add( A,M,1'b0,sum);
endmodule
module ALU(b,a,cin,sum);
        input [7:0] a,b;          input  cin; output  [7:0]sum;
             assign sum = a + b + cin;
endmodule
```

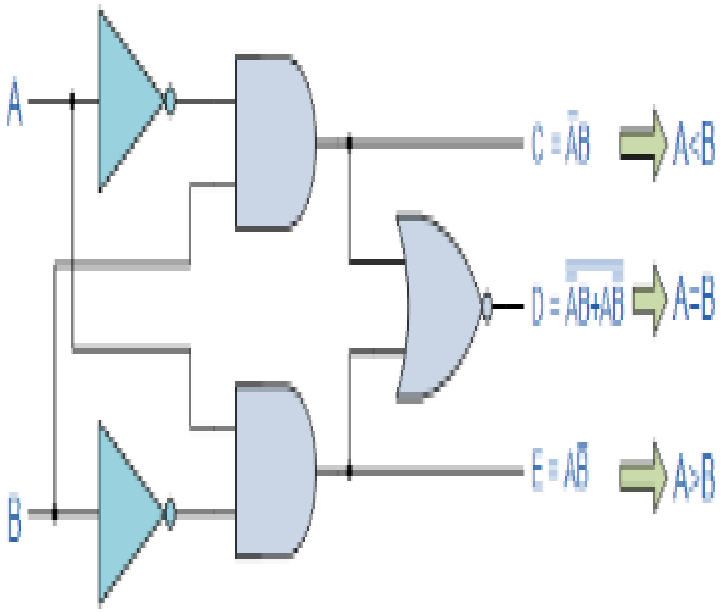
```
module multiplier(prod, busy, mc, mp, clk, start);
output [15:0] prod;output busy;input [7:0] mc, mp;input clk, start;
reg [7:0] A, Q, M;reg Q_1;reg [3:0] count; wire [7:0] sum, difference;

always @(posedge clk)
begin
if (start)  begin
            A <= 8'b0;M <= mc;Q <= mp;Q_1 <= 1'b0;count <= 4'b0;
endelse  begin
 case ({Q[0], Q_1})
                        2'b0_1 : {A, Q, Q_1} <= {sum[7], sum, Q};
                        2'b1_0 : {A, Q, Q_1} <= {difference[7], difference, Q};
                        default: {A, Q, Q_1} <= {A[7], A, Q}; endcase
                        count <= count + 1'b1; end end
            alu adder (sum, A, M, 1'b0); alu subtracter (difference, A, ~M, 1'b1);
            assign prod = {A, Q}; assign busy = (count < 8);
endmodule
//The following is an alu.//It is an adder, but capable of subtraction:// subtraction means adding the two's complement--//a - b = a + (-b) = a + (inverted b + 1)//The 1 will be coming in as cin (carry-in)
module alu(out, a, b, cin);
            output [7:0] out; input [7:0] a;input [7:0] b;input cin;
            assign out = a + b + cin;
endmodule
```

Testbench for Booth's Multiplier

```
module testbench;
  reg clk, start;reg [7:0] a, b;
  wire [15:0] ab;
   wire busy;
   multiplier multiplier1 (ab, busy, a, b, clk, start);
  initial begin clk = 0;
        $display("first example: a = 3 b = 17");a = 3; b = 17; start = 1; #50 start = 0;#80
        $display("first example done");
        $display("second example: a = 7 b = 7");a = 7; b = 7; start = 1; #50 start = 0;#80
        $display("second example done");
        $finish;
end
        always #5 clk = !clk;
           always @(posedge clk)
                $strobe("ab: %d busy: %d at time=%t", ab, busy,$stime);
endmodule
```

1 bit Comparator

Inputs		Outputs		
B	A	A > B	A = B	A < B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0



2 bit Comparator

INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

A>B:A1B1’ + A0B1’B0’ + A1A0B0’

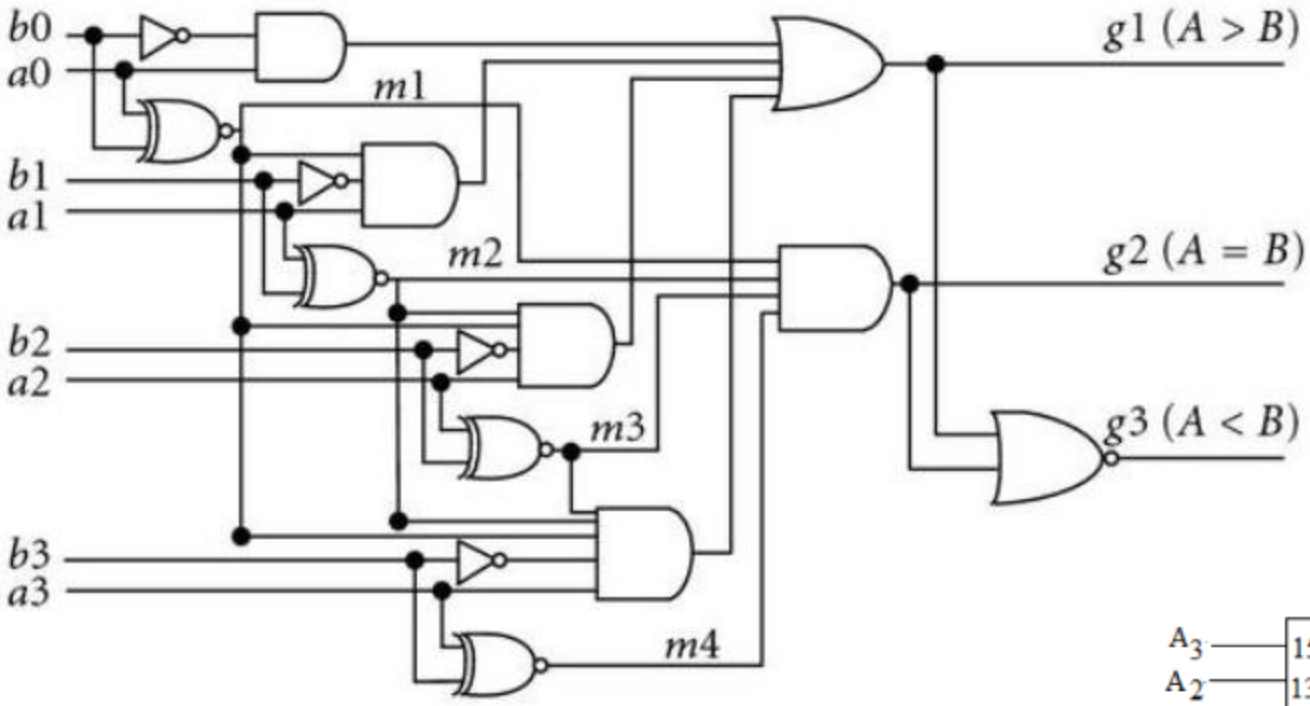
A=B: A1’A0’B1’B0’ + A1’A0B1’B0 + A1A0B1B0 + A1A0’B1B0’ : A1’B1’ (A0’B0’ + A0B0) + A1B1 (A0B0 + A0’B0’) : (A0B0 + A0’B0’) (A1B1 + A1’B1’) : (A0 ⊙ B0) (A1 ⊙ B1)

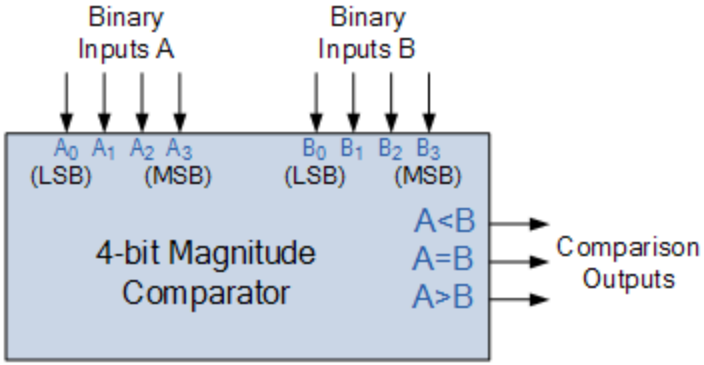
A<B:A1’B1 + A0’B1B0 + A1’A0’B0

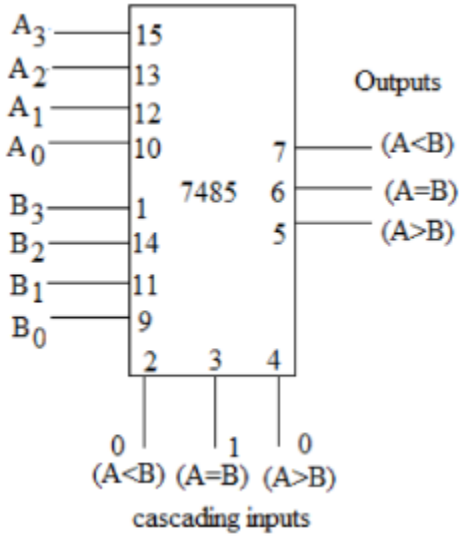
A=B

- The condition of A=B is possible only when all the individual bits of one number exactly coincide with corresponding bits of another number.
- **A=B: $A_1'A_0'B_1'B_0' + A_1'A_0B_1'B_0 + A_1A_0B_1B_0 + A_1A_0'B_1B_0$‘**
  : $A_1'B_1' (A_0'B_0' + A_0B_0) + A_1B_1 (A_0B_0 + A_0'B_0')$
  : $(A_0B_0 + A_0'B_0') (A_1B_1 + A_1'B_1')$
  A=B: $(A_0 \odot B_0) (A_1 \odot B_1)$

4-Bit Magnitude comparator







A>B

- In a 4-bit comparator the condition of A>B can be possible in the following four cases:

- If A3 = 1 and B3 = 0
- If A3 = B3 and A2 = 1 and B2 = 0
- If A3 = B3, A2 = B2 and A1 = 1 and B1 = 0
- If A3 = B3, A2 = B2, A1 = B1 and A0 = 1 and B0 = 0

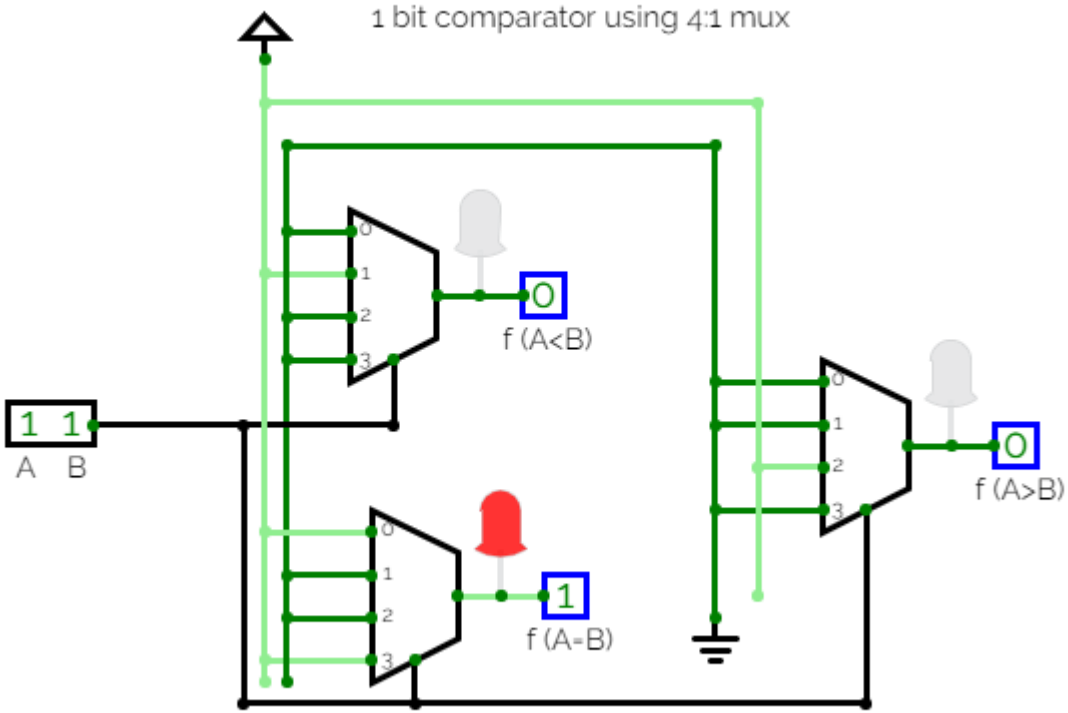
A<B

- Similarly the condition for A<B can be possible in the following four cases:
- If A3 = 0 and B3 = 1
- If A3 = B3 and A2 = 0 and B2 = 1
- If A3 = B3, A2 = B2 and A1 = 0 and B1 = 1
- If A3 = B3, A2 = B2, A1 = B1 and A0 = 0 and B0 = 1

1 bit Comparator verilog code

```
module comp_1bit(a,b,lt,eq,gt);
    input a,b;
     output lt,gt,eq;
        wire abar,bbar;
            assign abar = ~a;
                        assign bbar = ~b;
            assign lt = abar & b;
                        assign gt = bbar & a;
            assign eq = ~(lt|gt);
endmodule
```

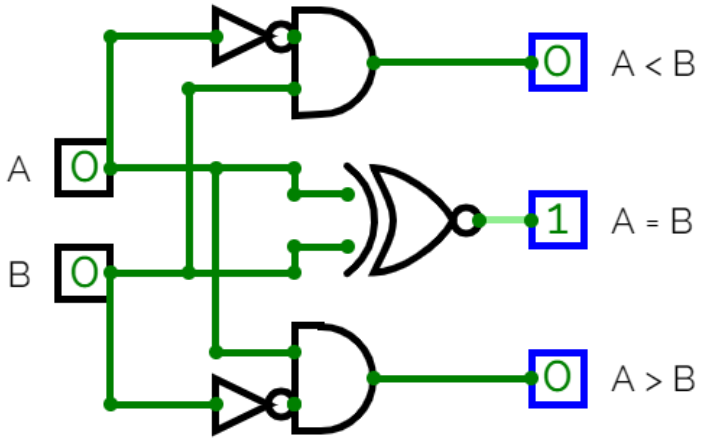
1 bit comparator using 4:1 MUX

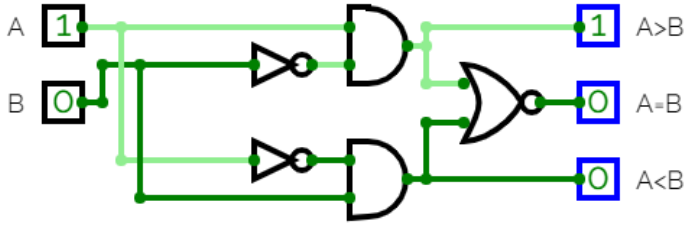


https://circuitverse.org/users/139620/projects/design-1-bit-comparator-using-4-1-mux-5315baa1-c06c-4495-83af-e0dcabc04a84

1 bit comparator

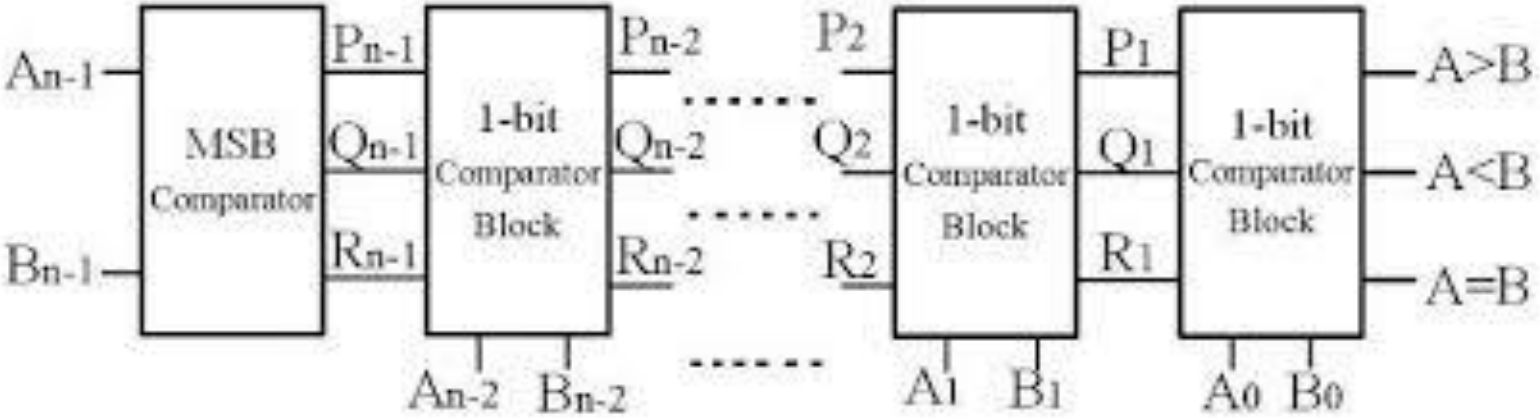
https://circuitverse.org/simulator/edit/comparator-1b4f8e41-fc1f-4d7a-8028-fb7803da64f3

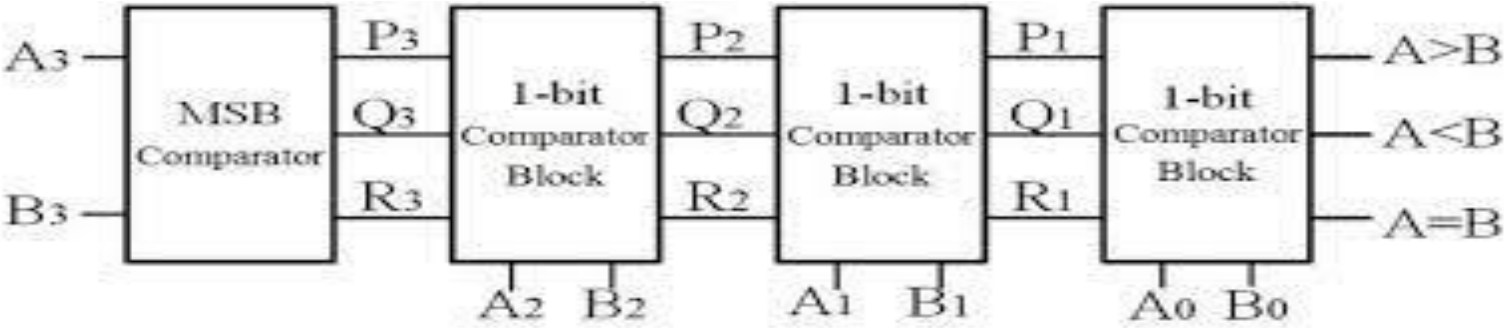




https://circuitverse.org/simulator/edit/1-bit-comparator-61c9165d-1870-4a14-8bb8-8cfa3cebc8e4

N bit and 4 bit comparator circuits





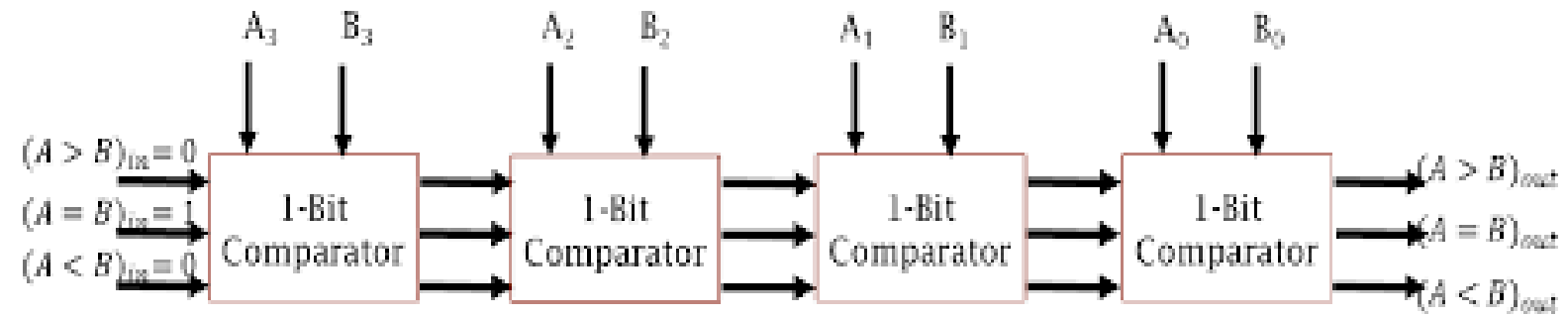


Fig 2: 4-Bit Comparator using 1-bit Cascaded Comparator

4 bit comparator Verilog code

```
module comparator4(A,B,LT1,GT1,EQ1,LT2,GT2,EQ2);
        input [3:0] A,B;
    output LT2,GT2,EQ2;
    input LT1,GT1,EQ1;
wire x30,x31,x32,x20,x21,x22,x10,x11,x12,x00,x01,x02;
wire x40,x41,x42,x50,x51,x52,x61,x62;
    comp_1bit c3(A[3],B[3],x30,x31,x32);
    comp_1bit c2(A[2],B[2],x20,x21,x22);
    comp_1bit c1(A[1],B[1],x10,x11,x12);
    comp_1bit c0(A[0],B[0],x00,x01,x02);
```

```
    assign x40 = x31 & x20;
        assign x41 = x31 & x21 & x10;
            assign x42 = x31 & x21 & x11 & x00;
                                assign x50 = x31 & x22;
                        assign x51 = x31 & x21 & x12;
                                                        assign x52 = x31 & x21 & x11
& x02;
                                assign EQ = (x31 & x21 & x11 & x01);
                                  assign EQ2 = EQ & EQ1;
                                    assign x61 = EQ & LT1;
                                     assign x62 = EQ & GT1;
                                   assign LT2 = (x30 | x40 | x41 | x42) | x61;
                              assign GT2 = (x32 | x50 | x51 | x52) | x62;
 endmodule
```

The Verilog code for the 16-bit Comparator

```
module comp16(a,b,lt1,gt1,eq1);
    input [15:0] a,b; output lt1,gt1,eq1;
       parameter eq =1'b1; parameter lt=1'b0;
            parameter gt=1'b0;
              wire t11,t12,t13,t21,t22,t23,t31,t32,t33;

      comparator4 c1(a[3:0],b[3:0],lt,gt,eq,t11,t12,t13);
     comparator4 c2(a[7:4],b[7:4],t11,t12,t13,t21,t22,t23);
    comparator4 c3(a[11:8],b[11:8],t21,t22,t23,t31,t32,t33);
    comparator4 c4(a[15:12],b[15:12],t31,t32,t33,lt1,gt1,eq1);
endmodule
```

REFERENCE

- https://technobyte.org/2-bit-4-bit-comparator/ VERILOG CODES FOR CLC:
- https://digitalsystemdesign.in/wp-content/uploads/2018/05/Combinational-Curcuits.pdf